加餐篇——OpenClaw v2026.3.31 最新版升级与新功能上手

§1 OpenClaw 一周四连发

如果你在 Ch1 中跟着我们搭建了 Cron 自动追踪工作流，那你的 OpenClaw 应该已经"主动报告"了好几次——从我们上次升级到 v2026.3.23-2 至今，不到两周时间，OpenClaw 又连续发布了四个版本。

没错，四个。这就是开源社区最活跃的 AI Agent 框架的更新节奏。

好消息是，这一轮更新不像 v2026.3.22 那次"大爆炸"——没有让 6 个插件集体失效的破坏性变更，没有 Dashboard 崩溃，也不需要清理旧环境变量。从 v2026.3.23 开始，OpenClaw 团队走的是渐进增强路线：修一批 bug、加一批功能、收紧一层安全——每个版本都在上一个的基础上稳步前进。

下面这张表让你一眼看清四个版本各自在做什么：

版本	发布日期	一句话定位	国内用户价值
v2026.3.23	03-23	稳定修复——给 3.22 擦屁股	中
v2026.3.24	03-25	功能增强——OpenAI 兼容网关 + 内置 Skill + /tools 改版	高
v2026.3.28	03-29	功能增强——MiniMax 图像生成 + 企业安全钩子	中高
v2026.3.31	03-31	当前最新稳定版——QQ Bot 内置 + 安全加固 + 后台任务统一	高

Platform Solutions Resources Open Source Enterprise Pricing Search or jump to... Sign in Sign up

openclaw / openclaw Public

Sponsor Notifications Fork 68k Star 343k

Code Issues 5k+ Pull requests 5k+ Actions Security and quality 427 Insights

Releases / v2026.3.31

openclaw 2026.3.31 Latest

Compare

steipete released this 7 hours ago · 111 commits to main since this release v2026.3.31 213a704

Breaking

- Nodes/exec: remove the duplicated `nodes.run` shell wrapper from the CLI and agent `nodes` tool so node shell execution always goes through `exec host=node`, keeping node-specific capabilities on `nodes invoke` and the dedicated media/location/notify actions.
- Plugin SDK: deprecate the legacy provider compat subpaths plus the older bundled provider setup and channel-runtime compatibility shims, emit migration warnings, and keep the current documented `openclaw/plugin-sdk/*` entrypoints plus local `api.ts` / `runtime-api.ts` barrels as the forward path ahead of a future major-release removal.
- Skills/install and Plugins/install: built-in dangerous-code `critical` findings and install-time scan failures now fail closed by default, so plugin installs and gateway-backed skill dependency installs that previously succeeded may now require an explicit dangerous override such as `--dangerously-force-unsafe-install` to proceed.
- Gateway/auth: `trusted-proxy` now rejects mixed shared-token configs, and local-direct fallback requires the configured token instead of implicitly authenticating same-host callers. Thanks @zhangning-agent, @jacobtomlinson, and @vincentkoc.
- Gateway/node commands: node commands now stay disabled until node pairing is approved, so device pairing alone is no longer enough to expose declared node commands. (#57777) Thanks @jacobtomlinson.
- Gateway/node events: node-originated runs now stay on a reduced trusted surface, so notification-driven or node-triggered flows that previously relied on broader host/session tool access may need adjustment. (#57691) Thanks @jacobtomlinson.

Changes

- ACP/plugins: add an explicit default-off ACPX plugin-tools MCP bridge config, document the trust boundary, and harden the built-in bridge packaging/logging

> GitHub Releases 页面 是查看 OpenClaw 每个版本完整变更日志的权威来源——但全英文、罗列式、每个版本几十项变更——这也是我们写这章加餐的意义所在。

本章你会得到什么

花不到 1 小时跟完这章，你会：

1. **升级成功**：从 v2026.3.23-2 一步到位升级到 v2026.3.31 最新版，Gateway 正常运行
2. **看懂更新**：获得一张四版本更新速查表，按"和我有关 / 了解即可"分类标注——不用再翻英文 Release Notes
3. **上手新功能**：亲手体验 5 个关键新功能——`/tools` 新界面、内置 Skill 安装、OpenAI 兼容端点、`openclaw config schema`、MiniMax 图像生成

阅读路径建议

- **急用**：直接跳到 §2 完成升级，再按需看 §4/§5 的功能实操
- **全面了解**：从头到尾顺序阅读，约 50 分钟
- **只想看速查表**：翻到 §7 直接查

---

§2 升级实操：从 v2026.3.23-2 到 v2026.3.31

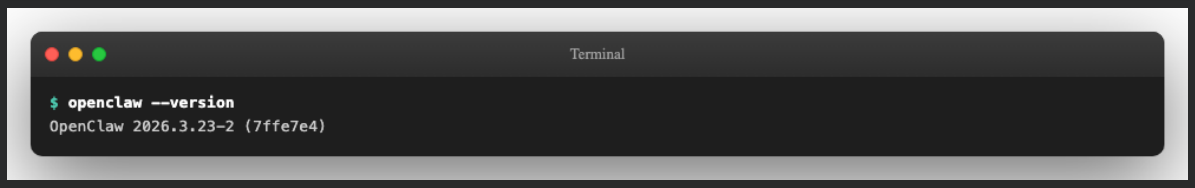
2.1 实操：npm 手动升级五步法

升级流程和 Ch1 完全一致——停 Gateway、装新版、跑 doctor、重启验证。这次跨了四个版本（3.23-2 → 3.23 → 3.24 → 3.28 → 3.31），但由于没有 v2026.3.22 那种破坏性变更，整个过程会比上次顺畅得多。

> 注：v2026.3.31 有 6 项 Breaking Changes，但大多面向插件开发者（Node exec 路由变更、Plugin SDK 迁移警告等）。普通用户通过 `doctor --fix` 即可自动处理。

Step 1：确认当前版本

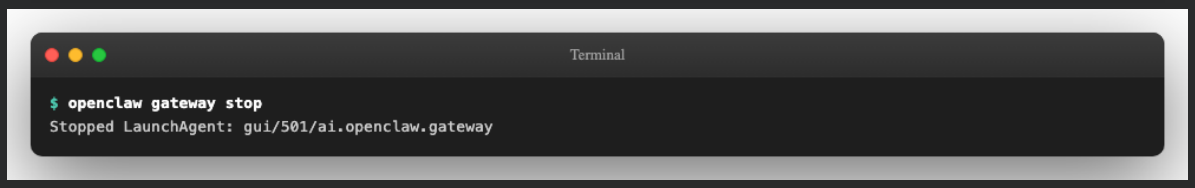
```
openclaw --version
```



确认输出 `OpenClaw 2026.3.23-2`，这是我们在 Ch1 升级后停留的版本。括号里的 `7ffe7e4` 是 Git commit 短哈希，用于精确定位代码版本。

Step 2：停止 Gateway

```
openclaw gateway stop
```



看到 `Stopped LaunchAgent: gui/501/ai.openclaw.gateway` 就说明 Gateway 已经干净退出。

> 重要提醒：为什么要先停 Gateway 再升级？因为 `npm install` 会替换 OpenClaw 的可执行文件和依赖库——如果 Gateway 正在运行，覆写过程可能导致进程崩溃（Issue #54790 记录了真实案例）。详细对比见 §2.2。

Step 3：安装最新版

```
npm install -g openclaw@latest
```

```
Terminal

$ npm install -g openclaw@latest
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead

added 23 packages, removed 258 packages, and changed 457 packages in 2m

88 packages are looking for funding
  run jiutian-v2
├── https://github.com/sponsors/ljharb
│   └── get-intrinsic@1.3.0, function-bind@1.1.2, gopd@1.2.0, has-symbols@1.1.0, has-tostringtag@1.0.2
└── https://github.com/sponsors/jimmywarting
    └── node-domexception@1.0.0 for details

$ openclaw --version
OpenClaw 2026.3.31 (213a704)
```

安装过程中你会看到类似 `added XX packages, removed XXX packages, and changed XXX packages` 的输出——具体数字因本地环境而异（我们的环境显示 added 23, removed 258, changed 457），只要没有红色 **`ERR!`** 错误就说明安装成功。那条 `node-domexception` 的 deprecation 警告可以忽略，不影响使用。

安装完成后立即确认版本：

```
openclaw --version
# 预期输出：OpenClaw 2026.3.31 (213a704)
```

看到 `2026.3.31` 就说明新版本已经就位。

> 注：这里我们用的是 `npm install -g openclaw@latest` 而不是 Ch1 中的 `npm update -g openclaw`。对于全局安装的包，两者在大多数情况下效果相同——但 `install @latest` 明确锁定最新稳定版标签，而 `update` 在部分 npm 版本中行为不一致（取决于安装方式），所以 `install` 更可靠。

Step 4：运行 doctor 自动修复

```
openclaw doctor --fix
```

```
Terminal

$ openclaw doctor --fix

┌  OpenClaw doctor
│
◇  State integrity
│  - Found 4 orphan transcript files. Doctor can archive them safely.
│
◇  Doctor changes
│  - Archived 4 orphan transcript files as .deleted timestamped backups.
│
◇  Security
│  - No channel security warnings detected.
│
◇  Skills status
│  Eligible: 16
│  Missing requirements: 41
│
◇  Plugins
│  Loaded: 43  Disabled: 41  Errors: 0
│
◇  Gateway service not installed.
│
Installed LaunchAgent: ai.openclaw.gateway.plist
│
└  Doctor complete.
```

`doctor --fix` 是升级后的"体检+修复"一体化命令。这次它做了两件事：

1. 归档孤立会话文件：发现 4 个不再被 `sessions.json` 引用的 `.jsonl` 文件，将它们重命名为 `.deleted` 后缀作为安全备份。这是正常的版本迁移清理。
2. 重新安装 **LaunchAgent**：因为 npm 升级替换了 OpenClaw 的二进制文件，macOS 的 LaunchAgent 配置需要重新注册才能指向新版本。

如果你看到 `Doctor complete.` 且没有红色错误提示，就说明一切正常。

Step 5：重启 Gateway 并验证

```
openclaw gateway start
openclaw status
```

```
Terminal

$ openclaw gateway start
Restarted LaunchAgent: gui/501/ai.openclaw.gateway

$ openclaw status
OpenClaw status

Overview
┌──────────────────┬──────────────────────────────────────────────────────┐
│ Item             │ Value                                                │
├──────────────────┼──────────────────────────────────────────────────────┤
│ Dashboard        │ http://127.0.0.1:18789/                              │
│ OS               │ macos 26.3.1 (arm64) · node 24.14.0                  │
│ Channel          │ stable (default)                                     │
│ Update           │ pnpm · up to date · npm latest 2026.3.31             │
│ Gateway          │ local · ws://127.0.0.1:18789 (local loopback)        │
│                  │ reachable 49ms · auth token                          │
│                  │ app 2026.3.31 macos 26.3.1                           │
│ Gateway service  │ LaunchAgent installed · loaded · running             │
│                  │ (pid 81124, state active)                            │
│ Agents           │ 1 · default main active                              │
│ Memory           │ enabled (plugin memory-lancedb)                      │
└──────────────────┴──────────────────────────────────────────────────────┘
```

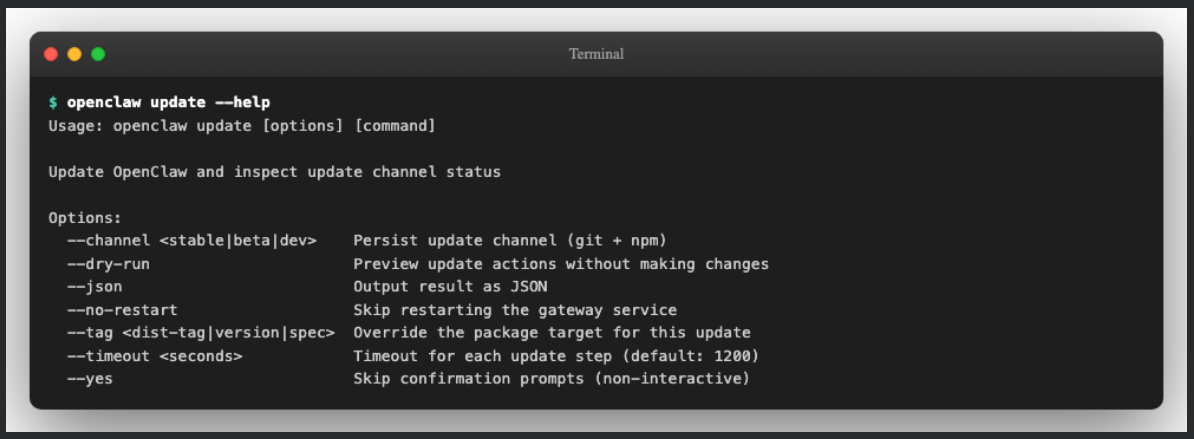
重点关注以下几项：

检查项	预期值	含义
Update	`up to date · npm latest 2026.3.31`	已是最新版，无需再更新
Gateway	`app 2026.3.31`	Gateway 运行的就是新版本
Gateway service	`running (pid xxxxx, state active)`	服务正常运行中

三项全部符合预期，升级完成。整个过程不到 5 分钟——比 Ch1 那次从 v2026.3.13 升级还要顺畅，因为这次没有破坏性变更需要处理。

2.2 升级方式对比：npm 手动 vs openclaw update

你可能注意到了，OpenClaw 其实提供了一个官方的一键升级命令——`openclaw update`。我们在 Ch1 和上面的实操中都选择了 npm 手动方式，这不是偶然。



两种方式的核心差异如下：

对比维度	**npm 手动方式**	**`openclaw update` 一键方式**
便捷性	需要 4-5 步操作	一条命令搞定
稳定性	高——手动停 Gateway 后再安装，避免运行中覆写	⚠️ 有已知崩溃风险（见下方说明）
可控性	高——每步可确认、可回退	低——黑盒操作，出错时不好定位
适用场景	大版本跨越 / 第一次升级 / 追求稳定	日常小版本更新 / 熟练用户
额外功能	无	支持 `--dry-run` 预览、`--channel` 切换

为什么我们推荐 npm 手动方式

`openclaw update` 确实更方便，但在实际使用中存在两个已知问题：

1. **Gateway 运行中覆写导致崩溃**（Issue #54790，截至 v2026.3.31 仍为 Open）：当 Gateway 正在运行时，`openclaw update` 内部调用 npm 安装新版本，覆写 `dist/` 目录的过程中，运

行中的 Gateway 进程读取到不完整的文件，导致崩溃。手动方式先停 Gateway 再安装，完全避免了这个问题。

2. **重复安装未变化的插件**（Issue #46955，截至 v2026.3.31 仍为 Open）：`openclaw update` 每次都会重新安装所有 npm 插件，即使它们没有任何变化，导致不必要的等待时间。

> 好消息：v2026.3.24 新增了 Node.js 版本预检——如果你的 Node.js 版本太低，`openclaw update` 会在开始安装前就给出明确的升级提示，而不是装到一半报错。现在升级到 v2026.3.31 后，后续如果只是日常小版本更新（如 3.31 → 3.32），`openclaw update` 会是更省事的选择。

总结：当前阶段推荐 npm 手动方式（稳定优先），等你对 OpenClaw 升级流程足够熟悉后，日常小版本更新可以切换到 `openclaw update`。

---

§3 v2026.3.23 变更速览：给 3.22 擦屁股

在讲新功能之前，先快速回顾一下 v2026.3.23——这个版本你已经在用了（当前环境就是 3.23-2）。它的主要工作是修复 v2026.3.22 大版本引入的问题：

变更	修了什么	对你的影响
插件运行时恢复	v2026.3.22 的构建变更导致 WhatsApp、Matrix 等多个插件的运行时文件未被打包进 npm 发布包，全局安装后这些插件直接无法加载	如果你用这些插件——现在它们能正常工作了
Qwen DashScope 标准端点	新增通义千问的按量计费 DashScope API 端点支持，Provider 组重命名为"Qwen (Alibaba Cloud Model Studio)"	🇨🇳 **国内用户利好**——通义千问接入更灵活，不再只有 Coding Plan 一种选择
ClawHub 凭证修复	macOS 上 ClawHub 登录状态在 Skill 搜索和 Gateway 浏览中丢失，降级为未认证模式	Skill 搜索结果现在和你的 ClawHub 账号匹配了
浏览器 **MCP** 可靠性	Chrome 会话附着后等待 Tab 可用再继续，而不是直接当作"已就绪"处理；修复了 Linux 环境下的二次启动回退问题	浏览器相关工具更稳定了
安全加固	CSP 指令改用 SHA-256 哈希白名单；Shell 包装命令拒绝单引号 Token 载体；Canvas 路由需要 Gateway 认证	安全性持续收紧——你不需要做任何操作
Mistral 模型修复	降低 Mistral 模型的默认 max-token 到安全范围，doctor 自动修复旧的 Mistral 配置	避免了 Mistral API 的 422 拒绝错误

> 注：v2026.3.23 有两个子版本（3.23-1 和 3.23-2），主要是 Plugin SDK 稳定性和自托管场景的渐进修复。我们当前使用的 3.23-2 已包含全部修复。

一句话总结：**v2026.3.23 让 3.22 能正常用了**。从这里开始，后续的 3.24、3.28、3.31 都是在稳定基础上的功能增强。

---

§4 v2026.3.24 重点功能：OpenClaw 迈向通用 AI 网关

4.1 Gateway OpenAI 兼容端点：让 OpenClaw 成为统一 AI 入口

OpenAI 兼容端点是 v2026.3.24 引入的标志性功能——它让 OpenClaw Gateway 对外暴露标准的 OpenAI API 接口（`/v1/models`、`/v1/chat/completions`、`/v1/embeddings`），这意味着任何支持 OpenAI SDK 的工具现在都能直接连接你的 Gateway。

为什么这很重要？想象一下：你在 OpenClaw 中配置好了 DeepSeek、通义千问、Moonshot 等多个模型——有了 OpenAI 兼容端点，LangChain、LlamaIndex、Open WebUI 这些第三方工具只需要把 `base_url` 指向 `http://127.0.0.1:18789`，就能通过 OpenClaw 统一调用所有已配置的模型，无需在每个工具中重复配置 API Key。

OpenClaw 不再只是一个聊天客户端——**它正在成为你本地的统一 AI 网关**。类似定位的开源工具还有 LiteLLM、Portkey 等——它们专注于模型路由和 API 统一。OpenClaw 的差异化在于：它不只是网关，还集成了 Agent 能力、多平台通道和 Skill 生态，是一个全栈 AI 助手平台。

> **Watch-out**：此端点默认关闭，需要在 `openclaw.json` 中手动启用。认证依赖 Gateway Token（Bearer Token），默认只监听 `127.0.0.1`（本机访问），不会暴露到外网。但如果你修改了监听地址使其对外可达，务必确保 Token 不被泄露——Token 泄露意味着任何人都能通过你的 Gateway 调用后端模型（并消耗你的 API 额度）。**注意 v2026.3.31 的变化**：此前 127.0.0.1 的本地调用会被隐式认证（不需要 Token），v2026.3.31 收紧后**本地调用也必须携带 Bearer Token**。如果升级后第三方工具连接 Gateway 出现 401 错误，请检查是否携带了 Token。

4.2 实操：curl 测试 /v1/models 端点

让我们动手验证 OpenAI 兼容端点是否正常工作。

Step 1：启用 OpenAI 兼容端点

用编辑器打开 OpenClaw 配置文件（推荐 VS Code：`code ~/.openclaw/openclaw.json`），在文件中找到 `"gateway"` 节点（如果没有就新建一个），添加如下配置：

```
{
  "gateway": {
    "http": {
      "endpoints": {
        "chatCompletions": { "enabled": true }
      }
    }
  }
}
```

> 注：如果你的 `openclaw.json` 中已经有 `"gateway"` 节点（比如包含 `auth` 配置），不要新建一个 `gateway`——在已有的 `gateway` 对象内添加 `"http"` 即可。JSON 不允许同名键。后续 §5.3 会介绍如何为这个配置文件启用自动补全——到时候再回来编辑会更从容。

保存后重启 Gateway 让配置生效：

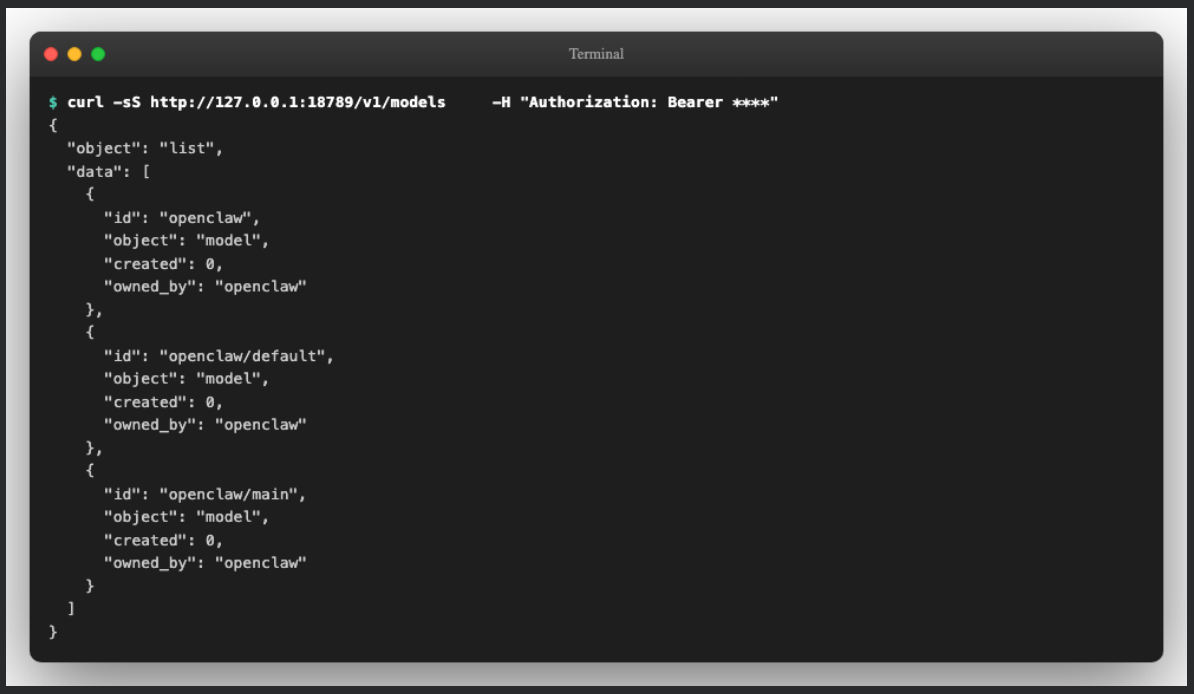
```
openclaw gateway stop
openclaw gateway start
```

> 注：启用 `chatCompletions` 后，`/v1/models` 端点会自动可用（它是模型发现的基础端点）。`/v1/embeddings` 需要额外配置 embeddings 模型后才能使用——这属于进阶用法，我们先跳过。另外，Gateway 的热重载不支持新增端点这种结构性变更，所以需要重启。

Step 2：curl 请求 /v1/models

```
curl -sS http://127.0.0.1:18789/v1/models \
    -H "Authorization: Bearer YOUR_TOKEN"
```

> 注：`YOUR_TOKEN` 是你的 Gateway 认证令牌，在 `~/.openclaw/openclaw.json` 的 `gateway → auth → token` 路径下，格式类似 `"token": "afb3c5..."`。快速提取命令：`grep -A1 '"token"' ~/.openclaw/openclaw.json`。如果你设置了代理，记得绕过本地地址：`curl --noproxy '*' ...`



返回结果是标准的 OpenAI `/v1/models` 格式，列出了三个可用的"模型"——实际上它们是 OpenClaw 的 Agent：

模型 ID	含义
`openclaw`	Gateway 默认入口
`openclaw/default`	默认 Agent
`openclaw/main`	名为 main 的 Agent（你配置的主 Agent）

当第三方工具（如 LangChain）发送请求到 `/v1/chat/completions` 并指定 `model: "openclaw/main"` 时，Gateway 会将请求路由到你的 main Agent，由该 Agent 使用其配置的后端模型（如 DeepSeek）来处理。

如果你想显式指定后端模型（绕过 Agent 配置），可以使用 `x-openclaw-model` 请求头——不过这属于进阶用法，日常使用中直接用 Agent 路由就够了。

> 深入理解：更多关于 OpenAI 兼容端点的配置选项（如 embeddings 支持、流式传输、模型路由规则），可参考官方文档 OpenAI Chat Completions。

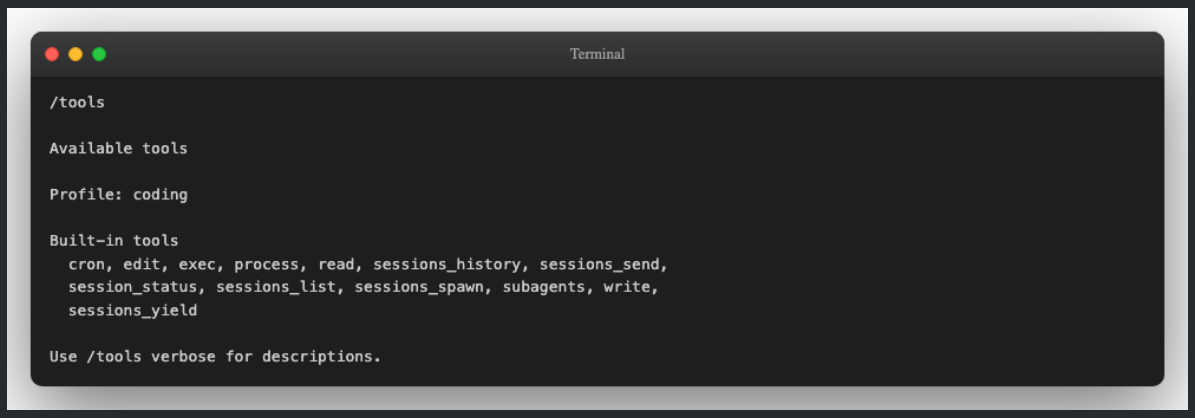
4.3 实操：/tools 新界面体验

v2026.3.24 重新设计了 `/tools` 命令——旧版会列出所有工具的完整描述，新版默认显示紧凑视图，只列出当前 Agent 实际可用的工具名称，一目了然。

Step 1：查看紧凑视图

在 TUI 或任何 Agent 会话中输入 `/tools`：

```
/tools
```



紧凑视图开头标注了当前的 **Profile**（`coding`），然后列出所有可用的内置工具名称。对比旧版（显示全部工具和完整描述、需要滚屏阅读），新版一屏就能看完——你能用什么工具，一眼了然。

Step 2：切换详细模式

需要看每个工具的详细说明时，输入 `/tools verbose`：

```
/tools verbose
```

```
Terminal

/tools verbose

Built-in tools
  Cron - Manage Gateway cron jobs (status/list/add/update/remove/run/runs)
  Edit - Edit a single file using exact text replacement.
  Exec - Execute shell commands with background continuation.
  Process - Manage running exec sessions: list, poll, log, write, send-keys.
  Read - Read the contents of a file. Supports text files and images.
  Session History - Fetch message history for a session.
  Session Send - Send a message into another session.
  Session Status - Show a /status-equivalent session status card.
  Sessions (sessions_list) - List sessions with optional filters.
  Sessions (sessions_spawn) - Spawn an isolated session.
  Subagents - List, kill, or steer spawned sub-agents.
  Write - Write content to a file.
  Yield - End your current turn.

Tool availability depends on this agent's configuration.
```

详细视图和旧版类似，列出每个工具的完整描述——当你想确认某个工具具体能做什么时使用。

> 实用建议：日常使用中 `/tools`（紧凑视图）就够了，只在需要查看工具详细说明时才用 `/tools verbose`。这个小改动减少了信息噪音，让你更快找到需要的工具。

4.4 实操：查看并体验内置 Skill

v2026.3.24 引入了 内置 **Skill**（**bundled skills**）——不需要从 ClawHub 下载，安装即用。运行 `openclaw skills` 可以看到完整列表：

```
openclaw skills
```

```
Terminal

$ openclaw skills
Skills (16/57 ready)

| Status        | Skill                | Description                                  | Source            |
|---------------|----------------------|----------------------------------------------|-------------------|
| ✓ ready       | coding-agent         | Delegate coding tasks to Codex, Claude Code  | openclaw-bundled  |
| ✓ ready       | gh-issues            | Fetch GitHub issues, spawn sub-agents        | openclaw-bundled  |
| ✓ ready       | tmux                 | Remote-control tmux sessions                 | openclaw-bundled  |
| ✓ ready       | weather              | Get current weather and forecasts            | openclaw-bundled  |
| △ needs setup | 1password            | Set up and use 1Password CLI                 | openclaw-bundled  |
| △ needs setup | apple-notes          | Manage Apple Notes via memo CLI              | openclaw-bundled  |
| △ needs setup | openai-whisper-api   | Transcribe audio via OpenAI Whisper API      | openclaw-bundled  |
| △ needs setup | session-logs         | Search and analyze session logs              | openclaw-bundled  |
| △ needs setup | trello               | Manage Trello boards, lists, and cards       | openclaw-bundled  |
| ... (57 skills total, 16 ready)                                                                |
```

注意列表中的三个状态标签：

状态	含义	你需要做什么
`✓ ready`	已就绪，可直接使用	无需操作
`△ needs setup`	需要额外配置（如 API Key 或 CLI 工具）	按提示完成配置
`disabled`	已禁用	在配置中启用

来源（Source）列区分了 `openclaw-bundled`（官方内置）和 `openclaw-extra`（额外安装）——本次新增的都是 `openclaw-bundled`。

值得关注的 Skill

`openclaw skills` 列出了 57 个可用 Skill（其中 7 个为 `openclaw-bundled` 官方内置，其余为 `openclaw-extra` 额外来源）。官方内置的这 7 个 Skill 开箱即用，不需要从 ClawHub 下载。对国内用户最实用的几个：

Skill	状态	用途	配置需求
`weather`	✓ ready	天气查询（via wttr.in/Open-Meteo）	零配置
`coding-agent`	✓ ready	委派编码任务给 Codex/Claude Code	零配置
`tmux`	✓ ready	远程控制 tmux 会话	零配置
`session-logs`	△ needs setup	搜索分析历史会话日志	配置日志路径
`apple-notes`	△ needs setup	管理 Apple Notes	安装 memo CLI

> **Watch-out**：v2026.3.31 收紧了第三方 Skill 的安装安全策略——如果安装从 ClawHub 下载的第三方 Skill 遇到 `critical` 级别的安全发现，系统会默认拒绝安装，需要加 `--dangerously-force-unsafe-install` 参数覆盖。内置 Skill 不受此限制。详见 §6.2 安全加固部分。

4.5 v2026.3.24 其他变更速览

除了上述重点功能外，v2026.3.24 还有这些值得一提的变化：

变更	简述	影响范围
Teams SDK 全面重构	Microsoft Teams 插件从旧版 Bot Framework 迁移到新架构	海外 Teams 用户
Docker/Podman 支持	新增 `--container` 参数，CLI 命令可以在容器内执行	容器化部署用户
Node.js 版本兼容降至 22.14+	不再要求最新 Node.js，向下兼容到 22.14	所有用户（降低入门门槛）
Discord 自动线程标题	Discord 频道中的对话自动生成线程标题	Discord 用户
Slack 富文本恢复	修复了 Slack 消息中富文本格式丢失的问题	Slack 用户
macOS 客户端侧边栏改版	桌面客户端的侧边栏 UI 重新设计	macOS 桌面客户端用户

---

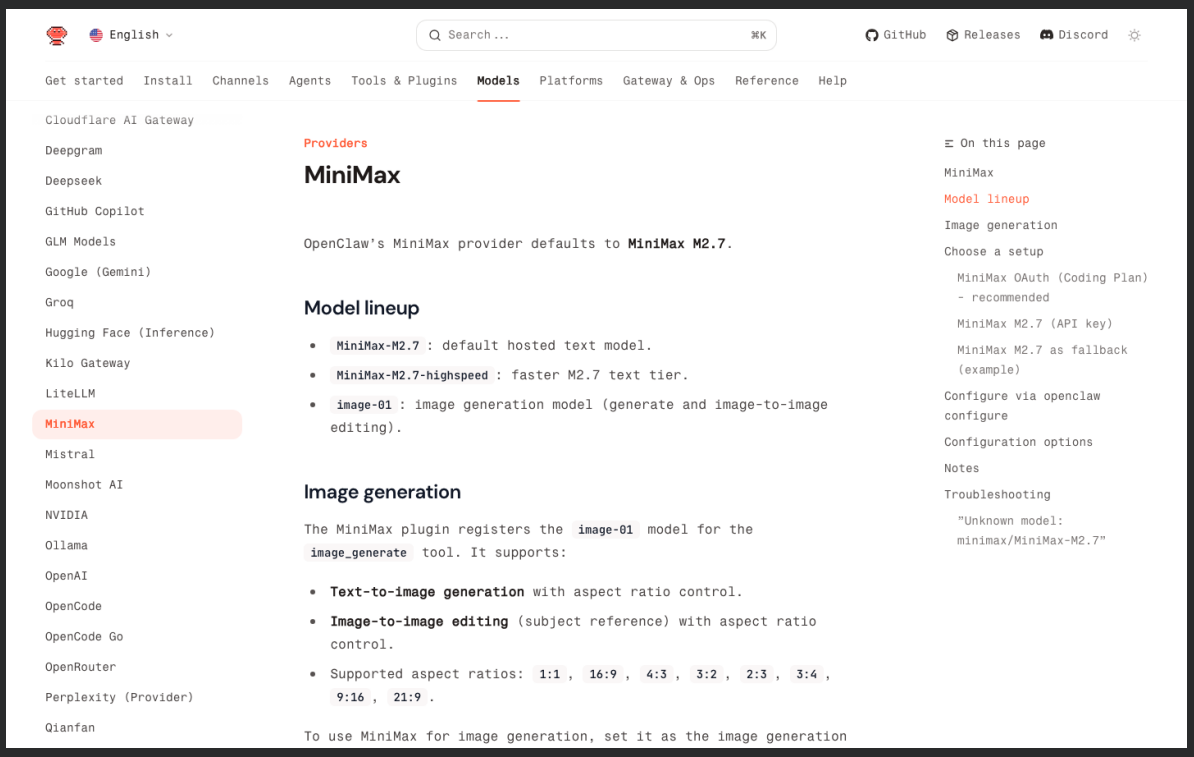
§5 v2026.3.28 重点功能：MiniMax 图像生成与企业安全

5.1 实操：MiniMax 图像生成——国内可直连的 AI 作图方案

MiniMax `image-01` 是 v2026.3.28 新增的图像生成模型——它让 OpenClaw 的 Agent 可以直接在对话中生成图片，而不需要借助外部工具。

为什么值得关注？三个理由：

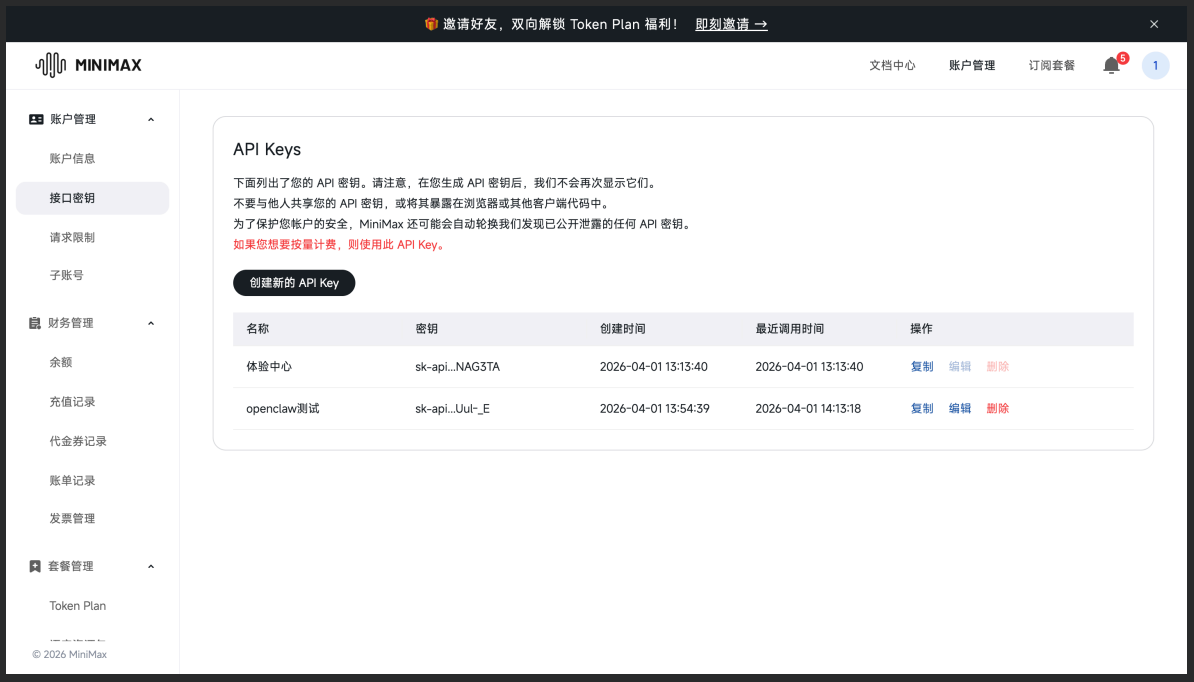
1. 国内可直连：MiniMax 是国内 AI 公司，API 服务在国内有节点，不需要翻墙就能用
2. 集成在 **Agent** 里：用自然语言告诉 Agent "帮我画一个…"，它会调用 `image_generate` 工具直接返回图片——无需离开 OpenClaw
3. 支持多种模式：文生图（text-to-image）、图到图编辑（image-to-image）、宽高比控制（1:1、16:9、4:3 等 8 种比例）



我们来实际走一遍：从注册获取 API Key，到配置 OpenClaw，到用自然语言让 Agent 画一张图——完整流程不到 5 分钟。

Step 1：获取 MiniMax API Key

前往 MiniMax 开放平台（注意域名是 `minimaxi.com`，多了个 `i`）注册账号（支持手机号注册）。登录后进入控制台，在左侧导航栏找到"接口密钥"菜单 → 点击"创建新的 API Key" → 复制生成的 API Key。新用户通常有免费额度。



API Key 格式为 `sk-api-` 开头的一长串字符（约 80+ 位），妥善保存——页面关闭后无法再次查看完整密钥。

Step 2：使用 onboard 命令一键配置

OpenClaw 提供了专门的 MiniMax 中国区 onboard 选项，一条命令搞定所有配置。

> ⚠️ 注意副作用：`onboard` 命令会覆盖你的默认模型配置——执行后所有对话将使用 MiniMax M2.7 而非你当前的模型（如 DeepSeek）。下面会告诉你怎么改回来，但请先知悉这一点。

```
openclaw onboard --auth-choice minimax-cn-api --minimax-api-key "你的-API-Key" --non-interactive --accept-risk
```

参数说明：`--non-interactive` 跳过交互式问答；`--accept-risk` 跳过 onboard 的风险提示确认框——这里的"风险"是指将 API Key 以明文存储在本地配置文件中，因为我们本来就需要保存 API Key，所以加上即可。

这条命令会自动完成三件事：

- 在 `models.providers` 中添加 MiniMax 提供商（baseUrl 指向 `api.minimaxi.com` 中国区端点）
- 注册 `image-01` 图像生成模型和 `MiniMax-M2.7` 文本模型
- 启用 MiniMax 插件（`plugins.entries.minimax`）

  改回默认模型：命令执行后，用编辑器打开配置文件（`code ~/.openclaw/openclaw.json`），搜索 `MiniMax-M2.7` 关键词，将 `agents.defaults.model.primary` 的值从 `"minimax/MiniMax-M2.7"` 改回你原来的模型（如 `"custom-api-deepseek-com/deepseek-chat"`）。只想用 MiniMax 画图、不想它接管对话，这一步不可省略。

  配置完成后，`openclaw.json` 中 MiniMax 相关部分如下（API Key 已脱敏）：

```
Terminal
{
  models: {
    providers: {
      minimax: {
        baseUrl: https://api.minimaxi.com/anthropic,
        apiKey: sk-api-iFK****l-_E,
        api: anthropic-messages,
        models: [
          {
            id: image-01,
            name: MiniMax
```

确认 `imageGenerationModel.primary` 设置为 `"minimax/image-01"` 后，重启 Gateway 使配置生效：

```
openclaw gateway stop && openclaw gateway start
```

> **Watch-out**：MiniMax 有两套 API 域名——`api.minimax.io`（国际站）和 `api.minimaxi.com`（中国站，注意多了个 `i`）。使用 `minimax-cn-api` 选项会自动配置中国站端点。如果你的 API Key 在 `minimax.io` 国际站注册，应改用 `minimax-global-api` 选项。域名选错会导致 `invalid api key` 错误。

Step 3：在 TUI 中生成图片

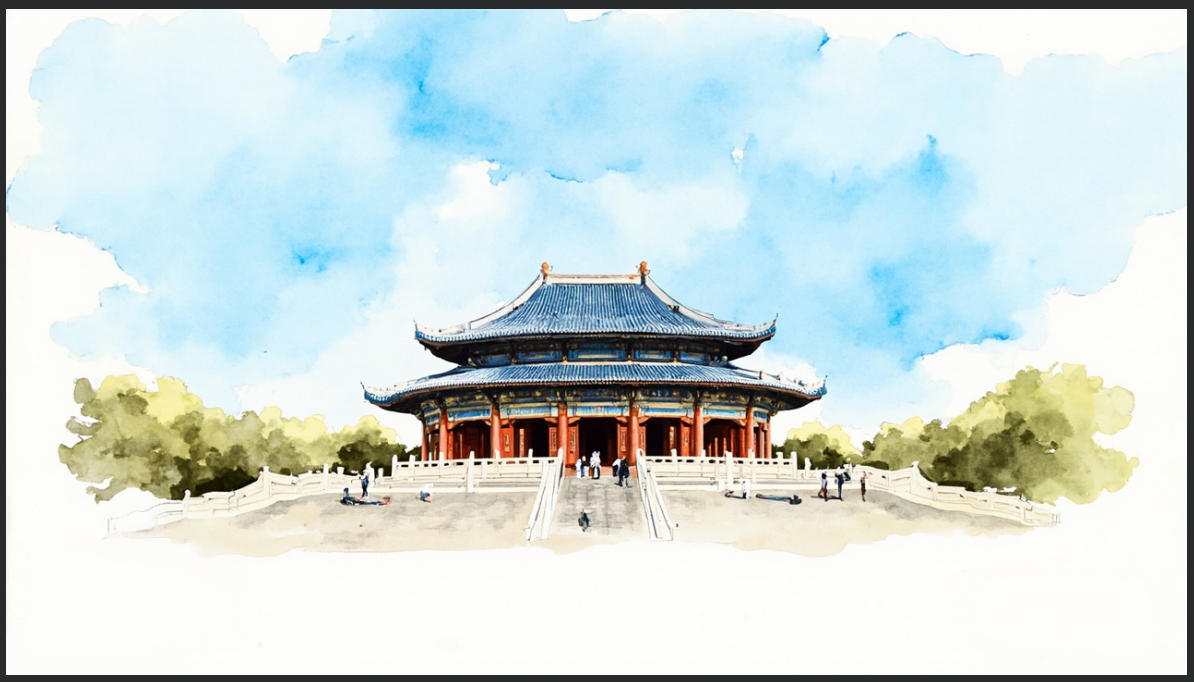
打开 TUI，用自然语言描述你想要的图片：

```
openclaw tui
> 请帮我画一张北京天坛祈年殿的水彩画，宽高比 16:9
```

Agent 接到请求后会自动调用 `image_generate` 工具，将你的描述转化为绘画提示词，发送给 MiniMax image-01 模型：

```
Terminal
/Users/hsail/.openclaw/completions/openclaw.zsh:3874: command not found: compdef
hsail@HsaildeMac-mini 一阶段测试v2 % openclaw tui

🦞 OpenClaw 2026.3.31 (213a704) — Built by lobsters, for humans. Don't question the hierarchy.

 openclaw tui - ws://127.0.0.1:18789 - agent main - session main

 session agent:main:main


 /tools


 Available tools

 Profile: coding

 Built-in tools
   cron, edit, exec, image_generate, process, read, sessions_history, sessions_send, session_status, sessions_list,
sessions_spawn,
 subagents, write, sessions_yield

 Use /tools verbose for descriptions.


 请使用image_generate工具，帮我生成一张北京天坛祈年殿的水彩画，宽高比16:9


 我将使用image_generate工具为你生成一张北京天坛祈年殿的水彩画，宽高比为16:9。已为你生成了一张北京天坛祈年殿的水彩画！图像已保存到媒体目录
中
 。

 这张水彩画展现了天坛祈年殿的经典建筑风格，采用了柔和的水彩色调和 艺术化的表现手法，符合16:9的宽高比要求。水彩画的特性让画面显
 得更加自然、有艺术感，蓝天白云的背景也增添了画面的美感。

 如果你想要调整风格、尺寸或者其他细节，我可以继续为你生成更多变体 。
 connected | idle
 agent main | session main (openclaw-tui) | custom-api-deepseek-com/deepseek-chat | tokens 15k/128k (12%)
```

图像生成完成后，文件自动保存到 `~/.openclaw/media/tool-image-generation/` 目录。打开看看效果：



从输入"画一张天坛水彩画"到拿到成品，Agent 在背后完成了提示词构建、API 调用、图片下载三步——对用户来说只需要一句话。

> 值得注意的细节：仔细看生成的图片——水彩风格渲染得不错，但这并不是祈年殿。真实的天坛祈年殿是圆形三重檐攒尖顶结构（中国建筑中最具辨识度的圆形建筑之一），而 AI 生成的是一座方形大殿。这提醒我们：当前 **AI** 图像生成在特定地标建筑的准确性上仍有局限——用于概念配图、氛围渲染完全没问题，但涉及真实建筑物或人物肖像时，务必人工核验结果。

> 实用建议：image-01 支持 8 种宽高比（1:1、16:9、4:3、3:2、2:3、3:4、9:16、21:9），可以在请求中指定。如果不指定，默认生成正方形（1:1）。其中 21:9 是超宽画幅，适合做横幅或封面。提示词用中文或英文都可以，但英文提示词通常能获得更细腻的效果。

三个常见踩坑

踩坑	现象	解决方法
域名选错	`invalid api key` 或 `login fail`	中国站注册的 Key 用 `minimax-cn-api`，国际站用 `minimax-global-api`
onboard 改了默认模型	对话突然变成 MiniMax M2.7 回复	手动改回 `agents.defaults.model.primary` 为你原来的模型
TUI 中看不到 **image_generate** 工具	`/tools` 列表中没有 `image_generate`	确认 `imageGenerationModel` 配置存在，重启 Gateway

> 注：关于 MiniMax M2.7 文本模型——如果你好奇"既然 onboard 装了 M2.7，要不要拿它当对话模型"，简短回答是：M2.7 是 MiniMax 的旗舰文本模型，中文对话能力不错，但与 DeepSeek V3 等国内头部模型相比仍有差距。如果你只需要 MiniMax 的图像生成能力，保持原来的默认模型是更明智的选择。

5.2 requireApproval 插件钩子：给 AI 加上"人在回路"安全阀

v2026.3.28 引入了 `requireApproval` 插件钩子——允许插件在 **AI Agent** 执行工具调用之前暂停执行，等待人类审批。

这解决了什么问题？ 在企业环境中，AI Agent 拥有执行命令、读写文件、发送消息等能力，但"自主行为不可控"一直是企业采用 AI Agent 的最大顾虑。`requireApproval` 提供了一个官方的"人在回路"（Human-in-the-Loop）机制——完整的 Hooks API 参考见 官方文档：

English Search ... ⌘K GitHub Releases Discord

Get started Install Channels Agents Tools & Plugins Models Platforms Gateway & Ops Reference Help

Skills
Skills
Creating Skills
Skills Config
Slash Commands
ClawHub
OpenProse

Automation
Automation Overview
Hooks
Standing Orders
Cron Jobs
Cron vs Heartbeat
Background Tasks
ClawFlow
Automation Troubleshooting
Webhooks
Gmail PubSub
Polls
Auth Monitoring

Automation

Hooks

Hooks provide an extensible event-driven system for automating actions in response to agent commands and events. Hooks are automatically discovered from directories and can be inspected with `openclaw hooks`, while hook-pack installation and updates now go through `openclaw plugins`.

Getting Oriented

Hooks are small scripts that run when something happens. There are two kinds:

- **Hooks** (this page): run inside the Gateway when agent events fire, like `/new`, `/reset`, `/stop`, or lifecycle events.
- **Webhooks**: external HTTP webhooks that let other systems trigger work in OpenClaw. See **Webhook Hooks** or use `openclaw webhooks` for Gmail helper commands.

Hooks can also be bundled inside plugins; see **Plugin hooks**. `openclaw hooks list` shows both standalone hooks and plugin-managed hooks.

Common uses:

- Save a memory snapshot when you reset a session

On this page
Hooks
Getting Oriented
Overview
Getting Started
Bundled Hooks
Onboarding
Trust Boundary
Hook Discovery
Hook Packs (npm/archives)
Hook Structure
HOOK.md Format
Metadata Fields
Handler Implementation
Event Context
Event Types
Command Events
Session Events
Agent Events
Gateway Events
Session Patch Events
Session Event Context
Example: Session Patch Logger Hook

维度	说明
工作原理	插件实现 `before_tool_call` 钩子，可以返回 `requireApproval`，Gateway 暂停执行并向用户弹出审批界面
审批形态	TUI 终端弹窗 / Telegram 内联按钮 / Discord 按钮 / Slack 原生审批路由（v2026.3.31 新增）
典型场景	高风险操作审批（如删除文件、执行系统命令、发送外部消息）
使用门槛	需要开发插件实现钩子——普通用户暂时关注概念即可

> **Watch-out**：`requireApproval` 目前需要自己写插件才能使用，还不是"开箱即用"的配置项。但作为 OpenClaw 官方推出的安全机制，预计未来会有更多内置的审批策略——关注这个方向没错。

从 Ch1 讲过的 ClawHavoc 安全事件（1,184 个恶意 Skill），到 v2026.3.28 的 `requireApproval` 钩子，再到 v2026.3.31 的 Slack 原生审批路由——OpenClaw 团队在持续构建企业级安全能力。

5.3 实操：openclaw config schema 与 IDE 自动补全

`openclaw config schema` 是 v2026.3.28 引入的一个小而美的功能——它输出 OpenClaw 配置文件的完整 JSON Schema，让你在 VS Code 等编辑器中获得自动补全和校验。

Step 1：导出 JSON Schema

```
openclaw config schema | head -20
```

```
Terminal

$ openclaw config schema | head -20
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "$schema": { "type": "string" },
    "meta": {
      "type": "object",
      "properties": {
        "lastTouchedVersion": { "type": "string" },
        "lastTouchedAt": { ... }
      }
    },
    "env": { ... },
    "gateway": { ... },
    "agents": { ... },
    "providers": { ... },
    ...
  }
}

$ openclaw config schema | wc -l
   32718
```

这条命令输出了一份 32,718 行的 JSON Schema——涵盖了 `openclaw.json` 配置文件中的每一个字段、每一个选项、每一个默认值。你不需要读完它，而是让编辑器来用它。

Step 2：在 VS Code 中配置自动补全

将 Schema 保存到本地文件，然后在 VS Code 的 `settings.json` 中关联：

```
# 导出 Schema 到文件
openclaw config schema > ~/.openclaw/openclaw-schema.json
```

```
Terminal

$ openclaw config schema > ~/.openclaw/openclaw-schema.json
$ wc -l ~/.openclaw/openclaw-schema.json
   32718 /Users/hsail/.openclaw/openclaw-schema.json

# VS Code settings.json 配置示例:
{
  "json.schemas": [
    {
      "fileMatch": ["**/openclaw.json"],
      "url": "file:///Users/hsail/.openclaw/openclaw-schema.json"
    }
  ]
}
```

在 VS Code 中打开 `settings.json`（`Cmd+Shift+P` → "Preferences: Open User Settings (JSON)"），添加：

先运行这条命令获取你的完整路径：

```
echo "file://$(realpath ~/.openclaw/openclaw-schema.json)"
# 输出类似: file:///Users/yourname/.openclaw/openclaw-schema.json
```

然后将输出粘贴到 VS Code 配置中：

```
{
  "json.schemas": [
    {
      "fileMatch": ["**/openclaw.json"],
      "url": "file:///Users/你的用户名/.openclaw/openclaw-schema.json"
    }
  ]
}
```

> 注：请务必将 `你的用户名` 替换为你的实际 macOS 用户名（即上面命令输出的路径）。VS Code 的 `json.schemas` 不支持 `~` 展开，必须使用完整绝对路径，否则 Schema 会静默失效——没有自动补全就说明路径没配对。

配置完成后，在 VS Code 中编辑 `openclaw.json` 时会自动获得：

- **键名自动补全**：输入 `"` 会提示所有可用的配置项
- **值类型校验**：输错类型会红线提示
- **悬停文档**：鼠标悬停在键名上显示说明

> 实用建议：这个配置一次性完成后就一劳永逸了——每次升级 OpenClaw 后重新运行 `openclaw config schema > ~/.openclaw/openclaw-schema.json` 更新即可。从此编辑 `openclaw.json` 不再需要翻文档猜配置项。

> 替代方案：如果你觉得手动配置 JSON Schema 有点繁琐，VS Code 市场上已有社区开发的 OpenClaw Config 扩展，安装后即可获得自动补全和校验。我们在这里教的是手动方式，好处是不依赖第三方扩展、可以精确控制 Schema 版本——每次升级 OpenClaw 后重新导出即可保证 Schema 与实际版本同步。扩展方式更省事，但 Schema 同步取决于扩展维护者的更新频率。

5.4 xAI/Grok x_search：海外用户的原生搜索方案

v2026.3.28 将 xAI 迁移到了 Responses API 架构，Grok 的网络搜索能力（`x_search`）现在原生内置——不再需要手动安装搜索插件。这是社区长期呼声的功能。

但对国内用户来说：**xAI API** 目前仅在北美和欧洲区域部署（如 **us-east-1**、**eu-west-1** 等），国内不可直连。如果你需要搜索功能，Ch1 中介绍的 9 大搜索 Provider（Moonshot、SearXNG 等）仍然是更实际的选择。

5.5 v2026.3.28 其他变更 + Qwen OAuth 迁移说明

变更	简述	你需要做什么
Qwen OAuth 废弃	通义千问的 OAuth 认证方式不再支持，需迁移到 Model Studio API Key 方式	如果之前用 OAuth 方式接入 Qwen，需要执行迁移（见下方说明）
旧配置迁移过期	两个月前的旧配置键名不再自动修复，`doctor` 会标记但不会自动处理	如果从很旧的版本升级，注意 doctor 的提示
Gemini 3.1 模型修复	修复了 Gemini 3.1 模型的解析问题	使用 Gemini 的用户自动受益
Telegram 消息分割	按词边界分割长消息，不再在单词中间截断	Telegram 用户体验改善
OpenRouter 图像回退	修复了 OpenRouter 图像分析的回退机制	使用 OpenRouter 的用户自动受益

Qwen OAuth 迁移操作

如果你之前通过 `portal.qwen.ai` 的 OAuth 方式接入通义千问，v2026.3.28 不再支持这种方式，需要迁移到 Model Studio API Key：

```
openclaw onboard --auth-choice modelstudio-api-key
```

执行后按提示输入你的 Model Studio API Key 即可完成迁移。如果你在 Ch1 中配置 Qwen 时就是用的 API Key 方式（而非 OAuth），则不需要做任何操作。

---

§6 v2026.3.31 重点功能：QQ Bot 内置与安全加固

v2026.3.31 是本章的升级终点——110 个 commits，6 项 Breaking Changes。体量不小，但主要集中在两个方向：国内平台扩展和安全持续加固。

6.1 QQ Bot 成为内置 Channel：国内即时通讯平台的里程碑

QQ Bot 正式成为 OpenClaw 的内置 Channel Plugin——这是 OpenClaw 第一次将国内即时通讯平台作为一等公民内置支持。

此前，国内用户如果想在即时通讯平台上使用 OpenClaw，可选方案是 Ch1 中介绍的飞书（通过 Extra Plugin）。QQ Bot 成为内置 Channel 意味着什么？

维度	之前（社区/外部插件）	现在（**v2026.3.31** 内置）
安装方式	自行寻找社区插件、手动安装	随 OpenClaw 一起安装，开箱即有
维护保障	社区维护，更新不确定	OpenClaw 团队官方维护，随版本迭代
功能完整度	因插件而异	多账号配置、SecretRef 凭证管理、斜杠命令、提醒、媒体消息
安全审计	不在 OpenClaw 安全审计范围	纳入官方安全审计流程

在 v2026.3.31 的安装包中，QQ Bot 的代码位于 `dist/extensions/qqbot/skills/`，包含三个子模块：

```
Terminal
$ ls ~/.npm-global/lib/node_modules/openclaw/dist/extensions/qqbot/
skills/

$ ls ~/.npm-global/lib/node_modules/openclaw/dist/extensions/qqbot/skills/
qqbot-channel/
qqbot-media/
qqbot-remind/
```

- `qqbot-channel`：核心通道能力——消息收发、斜杠命令
- `qqbot-media`：媒体消息支持——图片、视频、音频的双向传输
- `qqbot-remind`：提醒功能——定时提醒和通知

> **Watch-out**：QQ Bot 虽然已经内置，但完整配置涉及 QQ 开放平台的应用申请和认证——这是一个独立的配置主题，超出本章范围。目前你需要知道的是：这个能力已经准备好了，当你需要在 QQ 群中接入 AI Agent 时，不需要再四处找社区插件。
>
> 关于网上的 **QQ Bot** 教程：你在搜索引擎上能找到大量"OpenClaw 接入 QQ"的教程——这些都是基于 v2026.3.31 之前的社区插件方式（如 `tencent-connect/openclaw-qqbot` 等）。内置 Channel 和社区插件的核心区别是：内置 Channel 由 OpenClaw 团队官方维护，随版本迭代升级，安全审计纳入官方流程。如果你之前已经用社区插件配好了 QQ Bot，仍然可以继续使用——但建议关注后续版本中内置 Channel 的配置指南，适时迁移。

6.2 Skills 安装安全加固：从开放到审慎

还记得 Ch1 中我们提到的 ClawHavoc 安全事件吗？安天 CERT 在 ClawHub 上发现了 1,184 个恶意 Skill。v2026.3.31 的安全加固就是对这一事件的直接回应：

变化：内置的危险代码扫描器（built-in dangerous-code scanner）如果在 Skill 安装过程中发现 `critical` 级别的安全风险，默认拒绝安装（fail closed），而不是之前的警告但继续安装（fail open）。

这意味着什么？

```
# 之前（v2026.3.28 及更早）
openclaw skills install some-skill
# → 扫描发现 critical 风险 → ⚠️ 警告 → 继续安装

# 现在（v2026.3.31）
openclaw skills install some-skill
# → 扫描发现 critical 风险 → ❌ 拒绝安装
# → 要继续？必须显式确认：
openclaw skills install some-skill --dangerously-force-unsafe-install
```

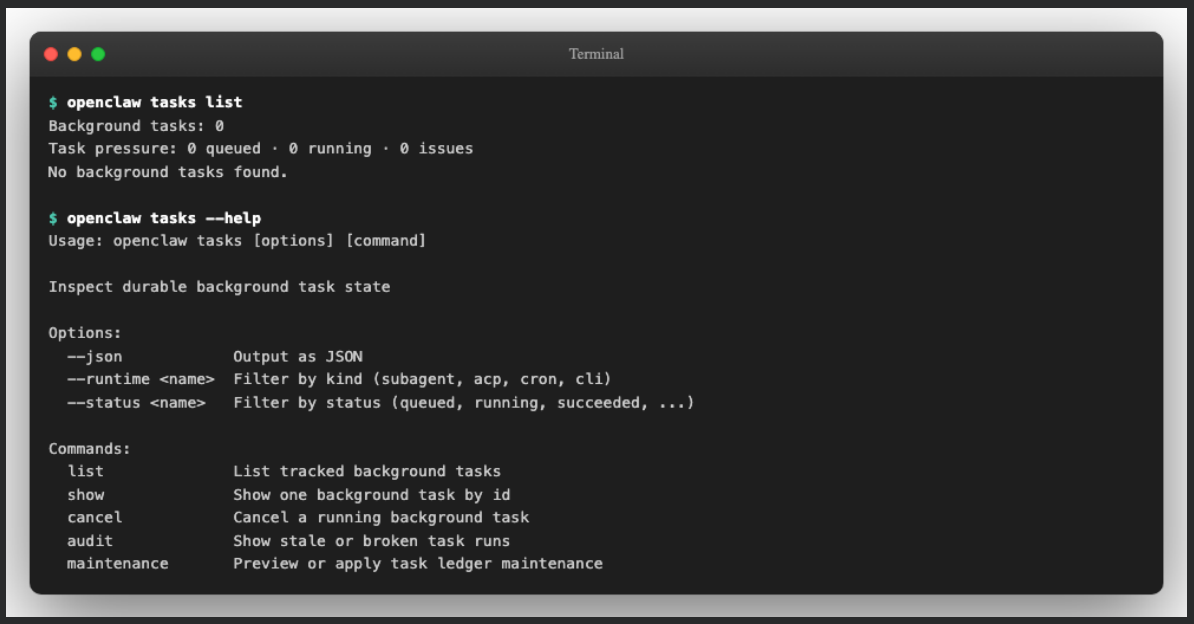
> 深入理解：`--dangerously-force-unsafe-install` 这个参数名故意设计得又长又醒目——OpenClaw 团队不想让你轻易跳过安全检查。如果你安装第三方 Skill 时遇到这个拦截，请先审查该 Skill 的代码，确认没有恶意行为后再使用 force 参数。内置 Skill（`openclaw-bundled` 来源）不受此限制。

从 Ch1 的 ClawHavoc 事件到 v2026.3.31 的默认拒绝策略，OpenClaw 的安全演进路径清晰可见：

```
发现问题（1,184 个恶意 Skill）→ 建立机制（dangerous-code 扫描器）→ 收紧策略（critical 默认拒绝）
```

6.3 v2026.3.31 其他核心变更速览

先看一个最直观的变化——`openclaw tasks` 命令，它是 v2026.3.31 统一后台任务管理的入口：



目前我们没有运行中的后台任务，所以显示"No background tasks found"。但一旦你启用了 Cron 定时任务（Ch1 中搭建的追踪工作流）、子 Agent 或 ACP 等功能，`openclaw tasks list` 就能统一查看它们的状态——不再需要去不同的地方检查不同类型的后台任务。

变更	简述	影响范围
统一后台任务控制面	ACP、子 Agent、Cron、后台 CLI 执行统一到 SQLite 记账本，新增 `openclaw tasks list/show/cancel` 命令管理所有后台任务	所有用户（更好的任务可见性）
Cron 会话隔离	Cron 任务跨重启保持 provider/model 选择，announce 模式保留完整多行报告——你在 Ch1 搭的 Cron 工作流现在更稳定了	Cron 用户
MCP 工具增强	工具名材料化为 `serverName__toolName` 格式；支持远程 HTTP/SSE MCP 服务器；新增 streamable-http 传输	MCP 用户
WhatsApp 表情反应	Agent 可以在 WhatsApp 消息上用表情符号回应	WhatsApp 用户
LINE 出站媒体	LINE 支持发送图片、视频和音频消息	LINE 用户
Matrix 流式回复	Matrix 消息支持草稿流式更新，部分回复更新同一消息而非发新消息	Matrix 用户
Slack 原生审批	执行审批路由直接在 Slack 内完成，无需跳转 Web UI	Slack 用户（呼应 §5.2）
安全加固（多项）	阻止 exec 环境中的代理/TLS/Docker 覆盖；可疑环境变量清理；ACP 语义审批替代危险工具覆盖	所有用户（安全基线提升）

关于 6 项 Breaking Changes

v2026.3.31 有 6 项 Breaking Changes，但绝大多数面向插件开发者，普通用户基本不受影响：

1. **Node exec 路由变更**：Shell 执行统一走 `exec host=node`，移除重复包装器
2. **Plugin SDK 迁移警告**：旧版 Provider 兼容路径废弃，抛出迁移警告
3. **Skills 安装安全收紧**：critical 发现默认拒绝（上文已详述）
4. **Gateway Auth 收紧**：拒绝混合的共享 Token 配置；本地直连（127.0.0.1）也需要显式携带 Bearer Token——此前同机调用会被隐式认证
5. **Node 命令审批**：Node 命令在 pairing 审批前保持禁用状态
6. **Node 运行受信面**：Node 发起的运行在缩减的受信表面上操作

如果你在 §2 的升级中 `doctor --fix` 没有报错，这些 Breaking Changes 已经被自动处理了。

---

§7 总结与速查

7.1 v2026.3.23→3.31 版本更新速查表

以下速查表综合了四个版本的英文 Release Notes，按国内用户价值重新排序和分类。据我们观察，这是目前中文社区唯一一份覆盖 **v2026.3.23→3.31 全部四版本增量的中文速查表**——建议收藏备查。

和我有关（日常使用会遇到）

功能	引入版本	对你意味着什么	国内可用
`/tools` 紧凑视图	v3.24	查看工具列表不再刷屏，一目了然	✅
内置 Skill（7 个 bundled + 50 个 extra）	v3.24	不用去 ClawHub 找，安装即用	✅
OpenAI 兼容端点	v3.24	LangChain/Open WebUI 能直连你的 Gateway 了	✅
`openclaw config schema`	v3.28	编辑 openclaw.json 有自动补全了	✅
MiniMax 图像生成	v3.28	Agent 可以直接画图，国内直连	✅
QQ Bot 内置 Channel	v3.31	你的 QQ 群能接 OpenClaw 了	✅
Skills 安装 critical 拒绝	v3.31	第三方 Skill 安装更安全了，可能遇到拦截	✅
Cron 会话隔离	v3.31	Ch1 搭的定时任务更稳定了	✅
`openclaw tasks` 命令	v3.31	查看和管理所有后台任务有统一入口了	✅
Qwen DashScope 端点	v3.23	通义千问接入更灵活了	✅
Qwen OAuth 废弃	v3.28	如果用 OAuth 接入 Qwen，需要迁移到 API Key	✅
Node.js 22.14+ 兼容	v3.24	不需要最新版 Node.js 也能跑 OpenClaw 了	✅

了解即可（特定场景或海外用户）

功能	引入版本	简述	国内可用
xAI/Grok x_search	v3.28	Grok 原生搜索，不需要搜索插件	❌ 国内不可直连
requireApproval 插件钩子	v3.28	AI 操作审批机制，需要开发插件	✅ 但需自行开发
Teams SDK 重构	v3.24	Teams 插件架构升级	⚠️ 国内使用率低
Docker/Podman `--container`	v3.24	CLI 命令在容器内执行	✅
WhatsApp 表情反应	v3.31	Agent 可以用表情回应消息	⚠️ 需代理
Slack 原生审批	v3.31	审批操作直接在 Slack 内完成	⚠️ 国内使用率低
MCP 远程 HTTP/SSE	v3.31	支持远程 MCP 服务器	✅
LINE 出站媒体	v3.31	LINE 发送图片/视频/音频	⚠️ 国内使用率低
Matrix 流式回复	v3.31	Matrix 部分回复更新同一消息	✅
Discord 线程标题	v3.24	频道对话自动生成标题	⚠️ 需代理
旧配置迁移过期	v3.28	两个月前的旧键名不再自动修复	✅
6 项 Breaking Changes	v3.31	主要面向插件开发者，doctor 自动处理	✅

7.2 本章回顾与持续跟进

回答开篇的两个问题：

> **EQ1**：从 **3.23-2** 升级到 **3.31**，会不会又像 **3.22** 那次踩坑？

不会。v2026.3.22 的两个严重 bug（Dashboard 崩溃、6 个插件失效）在 v2026.3.23 就已修复。从 3.23 到 3.31 走的是渐进增强路线，没有 3.22 那种大规模破坏性变更。我们的升级过程不到 5 分钟就完成了，`doctor --fix` 只做了常规清理。

> **EQ2**：这四个版本的新功能，哪些我现在就能用上？

上面的速查表"和我有关"列已经回答了——12 项功能直接可用，最值得马上试的是 `/tools` 新界面、内置 Skill、和 `openclaw config schema`。

五个要点回顾

1. 升级平稳：npm 五步法依然可靠，`openclaw update` 作为官方方案正在改进中

2. **OpenAI 兼容网关**：OpenClaw 正在从聊天桥梁走向通用 AI 网关，`/v1/models` 端点是关键标志
3. **MiniMax 图像生成**：国内直连的 AI 作图方案已就位——`minimax-cn-api` 一条命令配置，TUI 中一句话生图
4. **QQ Bot 内置**：国内即时通讯平台首次成为一等公民内置 Channel
5. **安全持续加固**：从 ClawHavoc 到 critical 默认拒绝，安全策略在不断收紧——这是好事

持续跟进

你在 Ch1 中搭建的 Cron 自动追踪工作流还在运行——v2026.3.31 的 Cron 会话隔离让它更可靠了（跨重启保持 provider/model 选择）。加上本章的升级流程，你已经建立了一套完整的 OpenClaw 更新节奏：

```
Cron 通知你有新版本 → npm 五步法升级 → 查速查表了解变了什么
```

OpenClaw 的更新速度不会放慢——但有了这套节奏，你不需要每天盯着 Release Notes，也能从容跟上。

本章内容截稿于 2026 年 4 月 1 日，基于 OpenClaw v2026.3.31。查看最新版本：`openclaw --version` 或访问 [GitHub Releases](#)。